TP 4 – Router & Composition API

Catalogue de livres (navigation multi-pages)

Pr. Ahmed Bentajer

Objectifs

- Configurer Vue Router : routes statiques, route dynamique, route 404
- Lire un paramètre de route et naviguer par programmation
- Repérer les limites de `props`/`$emit` entre deux vues non liées
- (Bonus) Réécrire un composant en Composition API avec `<script setup>`

Mise en place

```
cd starter
npm install
npm run dev
```

Pour info

Un fichier `src/store/books.js` est fourni, déjà fonctionnel. Il remplace le `data() { books }` de `App.vue` du TP3 : la page d'accueil et la page de détail d'un livre sont maintenant deux vues *différentes*, sans lien parent/enfant, donc `props`/`$emit` ne suffisent plus à partager `books` entre elles. Le fichier utilise `reactive()` pour créer un état partagé au niveau du module – lisez son en-tête, qui explique le principe. C'est une solution suffisante pour ce TP ; sur un projet plus grand, cette responsabilité est généralement confiée à **Pinia**, qu'on pourra introduire pour le mini-projet.

1 Configurer les routes

Complétez `src/router/index.js` : importez `createRouter` et `createWebHistory` depuis `vue-router`, ainsi que les trois vues fournies dans `src/views/` (`HomeView`, `BookDetailsView`, `NotFoundView`). Déclarez le tableau `routes` puis exportez le routeur créé.

2 Brancher la page d'accueil

Dans `src/views/HomeView.vue`, écoutez l'évènement `add-book` de `<AddBookForm>` pour appeler `addBook`, et passez `:books="store.books"` à `<BookList>` en branchant `@toggle-fav` et `@delete-book`. Complétez `addBook` pour générer un `id` et pousser le nouveau livre dans `store.books`.

3 Retrouver un livre par son id

Dans `src/views/BookDetailsView.vue`, complétez la propriété calculée `book` : elle doit retourner, dans `store.books`, le livre dont l'`id` correspond au paramètre de route.

Piège classique

`$route.params.id` est **toujours une chaîne de caractères**, même si l'URL ne contient que des chiffres (`/livre/2` donne `"2"`, pas `2`). Or les `id` dans `store.books` sont des nombres. Une comparaison `book.id === $route.params.id` échouera donc systématiquement – avec `===` en JavaScript, `2 === "2"` vaut `false`. Convertissez explicitement avec `Number(...)` avant de comparer.

4 Navigation programmatique

Complétez la méthode `retour` du bouton « Retour » pour qu'elle utilise `this.$router.back()` plutôt qu'un `<router-link>` – contrairement au lien « Voir détails » (déjà en place dans `BookList.vue`), ce bouton illustre la navigation déclenchée depuis du code plutôt que depuis un clic sur un lien.

Bonus – Composition API

Complétez `src/components/BookCardSetup.vue`, une réécriture de `BookCard.vue` en `<script setup>` : mêmes props (avec le même `validator`), mêmes évènements émis, plus une propriété calculée `libelleFavori` (« Retirer des favoris » / « Ajouter aux favoris ») utilisée comme texte du bouton.

Les quatre TP couvrent l'ensemble du support Vue.js et forment un catalogue de livres fonctionnel : base solide pour discuter du mini-projet (par exemple : recherche, pagination, persistance avec Firebase ou une API REST, migration du store maison vers Pinia).